221-2327 M.Rohaan Sec-E

Q#01:-

Sol.

Main Idea:-

* Use alternating Bit Protocol (1-Bit Sequence # 0 or 1)
* Each Packet and ACK has a checksum.
* Receiver only sends ACK, never NAK.
* Sender uses timer to resend on timeout.

1) Sender

States

* Wait for Data:- Ready to send new Data
* Wait for ACK:- Waiting for ACK of sent data.

Actions:-

* When data from app → send packet {seq, data, checksum}, start timer → go to Wait for ACK.
* When ACK received:
  → If correct and ACK number = current seq → stop timer, change seq (0→1, or 1→0), goto Wait for Data.
* If ACK wrong or corrupted → ignore.

CamScanner

* On timeout → resend same packets.

2) Receiver

States

* Wait for 0 or Wait for 1

Actions:

* if packet corrupted → ignore, resend last ACK
* if packet seq = expected → deliver data to app, send ACK(seq), change expected bit.
* if packet seq ≠ expected → duplicate → don't deliver, resend last ACK.

3) ACK Contents

* ACK includes the sequence number of the last correctly received packets.

4) Avoid Duplicates

* if ACK is lost, sender resends same packet.
* Receiver only delivers when packet's seq = expected, so duplicates are not delivered again.

CamScanner

5) Proof (At most once Delivery)

* Receiver Delivers data only once for each ~~stop~~ seq.
* After delivery, it flips expected bit, so any repeated packet with seq is ignored.

→ Each message delivered once only.

Q#02:-

Sol.

1) One-Way propogation delay and RTT

one-way delay = distance/speed = 2500 km / 2×10^8 m/s

= 0.0125 s = 12.5 ms

RTT = 2x one way = 25 ms.

2) Transmission time (L/R)

$L/R = 12000 / 10 \times 10^9 = 1.20 \times 10^{-6}$ s = 1.20 μs.

3) Sender Utilization (stop and wait)

$$U = \frac{L/R}{RTT + L/R} = \frac{1.20 \times 10^{-6}}{0.025 + 1.20 \times 10^{-6}} \approx 4.8 \times 10^{-5}$$

$U \approx 0.00480$ %

4) Factor increase with a 64-packet pipeline

$U_{64} \approx 64 \times 4.8 \times 10^{-5} = 3.07 \times 10^{-3}$

$= 0.307\%$

Factor increase $= U_{64} / U = 64$

Q#03:-

Sol:-

Setup:

* ISN_A = 10,000
* each segment carries 1000 bytes.
* Segments (by seq) cover bytes
  - → S1 : 10000 - 10999
    S2 : 11000 - 11999
    S3 : 12000 - 12999
* Receiver sends cumulative ACK = next expected byte (no SACK)
* Before any data, next expected byte ISN_A = 10000.

Process arrivals in order: second, third then first.

Arrival 1 second segment (S2)

* Receiver was expecting 10000, so S2 is out of order. It buffers S2 but cannot advance.
* Highest continuous byte received: none → ISN_A - 1 = 9999

CamScanner

* Cummulative ACK sent: next expected byte = 10000

Arrival 2 third segment (S3)

* still missing the first segment (10000 - 10999)
  Receiver buffers S3 too.
* Highest contiguous byte received: still 9999
* Comulative ACK sent: 10000

Arrival 3 first segment (S1)

* Now receiver has S1 + buffered S2 and S3
  → contiguous bytes 10000 - 12999.
* Highest contiguous byte received: 12999
* Comulative ACK sent: next expected byte
  = 12999 + 1 = 13000.

Q#04:-

Sol.

Given:-

one-way delay = d
negligible serialization, Window W = 4
timeout T = 4d.
Packets P0...P5 (6 Packets).
P1 and P3 are lost.

CamScanner

ACK0 and ACK2 are delayed by 3d

Go-Back-N (GBN)

Count transmissions:-

* initial sends: P0, P1, P2, P3 = 4
* retransmit after timeout: P1, P2, P3, P4 = 4
* send P5 once = 1

Total data transmission = 4 + 4 + 1 = 9

GBN final in-order delivery time = 7d.

Selective Repeat (SR)

What sender does: sends P0 - P3 at t=0
each packet has its own timer: T = 4d

* initial sends: P0, P1, P2, P3 = 4
* retransmit (individual): P1 and P3 = 2
* first send of P4 = 1
* first send of P5 = 1

Total data transmission = 4 + 2 + 1 + 1 = 8

SR final in-order delivery time = 7d

CamScanner

Q#05:-

Sol.

Step	Pkt Arr.	Rec. State	Sent ACKs	Deliver
1	P0	{0} → after delivering becomes {1}	ACK0	P0
2	P2 (P1 lost)	{2}	ACK2	—
3	P4 (P3 lost)	{2,4}	ACK4	—
4	retransmitted P1	{1,2,4} → deliver 1 then 2 → remaining {4}	ACK 1 (and now 2 is delivered so effectively ACK2 already sent earlier)	P1, P2
5	retransmitted P3	{3,4} → deliver 3 then 4 → {}	ACK 3 (ACK 4 already sent earlier)	P3, P4

* Receiver delivers only when it has the next in order packet.
* Out of order packet (2 and 4) are buffered untill missing packets (1 and 3) arrives.
* ACKs are sent for every correctly received packet (even if out of order).

CamScanner